ADVANCED PROGRAMMING LANGUAGES

Spring 2026, CM 0632. MSc in Computer Science and Information Technology, Ca' Foscari University of Venice

Parametric Types and Variance — Class Challenge Set, Solutions

Part A — Reading variance from the domain

1. `Comparator[A]`

`A` appears only in **input** position of `compare`. Natural variance: **contravariant**.

```
trait Comparator[-A]:
  def compare(x: A)(y: A): Int
```

Substitution check: a `Comparator[Animal]` knows how to compare any two animals, so it can be used wherever a `Comparator[Cat]` is required — `Comparator[Animal] <: Comparator[Cat]`. The reverse fails: a `Comparator[Cat]` cannot compare arbitrary animals.

2. `Producer[A]`

`A` appears only in **output** position of `next`. Natural variance: **covariant**.

```
trait Producer[+A]:
  def next: A
  def hasNext: Boolean
```

A `Producer[Cat]` yields `Cat`s, each of which is an `Animal`, so it fulfils the contract of `Producer[Animal]` — `Producer[Cat] <: Producer[Animal]`.

3. `Cache[K, V]`

- `K` appears only in input position (in `get` and `put`) → contravariant.

- `V` appears in *both* output (`get` returns `Option[V]`) and input (`put` takes a `V`) → must be **invariant**.

```
trait Cache[-K, V]:
  def get(key: K): Option[V]
  def put(key: K, value: V): Unit
```

Try `Cache[-K, +V]` : the compiler rejects `value: V` because `V` is then in a positive position but used in a negative one. Try `Cache[-K, -V]` : it rejects the return type `Option[V]` symmetrically. Invariance for `V` is forced.

4. The unsoundness witness for covariant `Comparator`

```
trait Comparator[+A]:                          // hypothetical: pretend the compiler accepts th
  def compare(x: A, y: A): Int

class IntComparator extends Comparator[Int]:
  override def compare(x: Int, y: Int): Int = x - y   // integer arithmetic, Int-specific

val ac: Comparator[Any] = new IntComparator    // OK under +A: Comparator[Int] <: Comparator[An
ac.compare("aaa", "abc")                       // typechecks: ac.compare expects (Any, Any) and
```

Statically the call is well-typed. At runtime dynamic dispatch resolves `compare` to `IntComparator.compare` , which performs `x - y` on values that are not `Int` s — a `ClassCastException` (or, depending on bridge generation, an `AbstractMethodError`).

The shape is identical to the `PowerQueue` argument from the lecture: a subclass specialises on a narrow parameter type and uses operations specific to it; covariance then lets sibling-typed arguments slip in through the wider static view, defeating the specialisation. Position calculus correctly flags the violation: in `Comparator[+A]` , the parameter `x: A` puts `A` in a *negative* position, which is incompatible with the declared positive variance.

Part B — Designing a callback registry

5. Variance of `Listener[E]`

`E` appears only in **input** position of `fire` : a listener *consumes* events. By LSP, a `Listener[Event]` (handles *anything*) can stand in wherever a `Listener[ClickEvent]` (handles only clicks) is required — being able to do more is being able to do less. Hence `ClickEvent <: Event` implies `Listener[Event] <: Listener[ClickEvent]` : **contravariance**.

```
trait Listener[-E]:
  def fire(event: E): Unit
```

Position calculus agrees: `E` in `fire(event: E)` sits in a negative position, consistent with the declared `-E` .

6. Concrete listeners and substitution

```
class ConsoleLogger extends Listener[Event]:
  def fire(event: Event): Unit = println(s"Event: $event")

class ClickCounter extends Listener[ClickEvent]:
  private var count = 0
  def fire(event: ClickEvent): Unit = count += 1

class KeyTracker extends Listener[KeyEvent]:
  private var keys: List[Int] = Nil
  def fire(event: KeyEvent): Unit = keys = event.code :: keys
```

With `Listener[-E]` :

```
val clicks = new Registry[ClickEvent]
clicks.subscribe(new ClickCounter)     // Listener[ClickEvent] — direct
clicks.subscribe(new ConsoleLogger)    // Listener[Event] <: Listener[ClickEvent] by contravaria
```

The second subscription is the payoff: a *single* generic logger object can be plugged into every specialised `Registry` . The user gets to write reusable, broad-coverage listeners (audit log, metrics, debug tracer) once and attach them anywhere, without writing per-event-type adapters. This is the typical contravariant gain in callback frameworks.

7. Variance of `Registry[E]`

Position calculus on `E` inside `Registry[E]`:

- `subscribe(l: Listener[E]): Unit` — the parameter is in negative position; inside `Listener[-E]`, `E` itself sits in a negative position. Negative × negative = **positive**. So this method puts `E` in a covariant position.
- `dispatch(e: E): Unit` — `e` is in negative position; `E` itself is straight in input. **Negative** position.

`E` is therefore used in *both* a positive and a negative position. `Registry[E]` must be **invariant**.

8. Adding `lastEvent: E`

`lastEvent: E` puts `E` in a positive (output) position — covariant. The variance picture does not change: we already had a covariant occurrence (through `subscribe`) and a contravariant one (through `dispatch`), so `Registry[E]` was already invariant and stays so. The only thing that would matter is if `Registry` had been variant *before* — for instance, if `dispatch` had been removed, leaving only `subscribe`, then `Registry` could have been declared `+E`, and `lastEvent` (also covariant) would not break it. As soon as both consuming and producing methods coexist on the same parameter, no variance annotation can be put on it.

Part C — Choosing the right design

9. Natural variance of `Serializer[A]`

`A` appears only in input position of `serialize`. **Contravariant.**

```
trait Serializer[-A]:
  def serialize(a: A): String
```

Reasoning: a more general serializer (one that knows how to serialize any `Animal`) certainly knows how to serialize a `Cat`, so `Serializer[Animal] <: Serializer[Cat]`.

10. The substitution payoff

With `Student <: Person <: Any` and `Serializer[-A]`:

Claim	Holds?	Why
`Serializer[Person] <: Serializer[Student]`	✓	`Student <: Person` and `-A` flips the order.
`Serializer[Student] <: Serializer[Person]`	✗	Would require `Person <: Student`.
`Serializer[Any] <: Serializer[Person]`	✓	`Person <: Any`, flipped by `-A`.
`Serializer[Any] <: Serializer[Student]`	✓	`Student <: Any`, flipped by `-A`.

A function `def f(s: Serializer[Student]): ...` accepts:

- `studentSerializer: Serializer[Student]` (trivially),
- `personSerializer: Serializer[Person]` (a serializer for the supertype handles the subtype),
- `objectSerializer: Serializer[Any]` (the universal fallback).

This is exactly what a library user wants: anywhere a `Student` serializer is asked for, the user may supply something *more general* and the library transparently uses it. Contravariance turns the type system into a search for the best (most specific) serializer the caller has on hand, with painless fallback to broader ones.

11. `def example: A` breaks variance

`example: A` is in output (positive) position. Together with `serialize`'s negative occurrence, `A` is now in both positions ⇒ `Serializer[A]` must become **invariant**, killing the substitution payoff of Challenge 11.

Fix using a lower bound (the same trick the lecture used on the queue):

```
trait Serializer[-A]:
  def serialize(a: A): String
  def example[B >: A]: B
```

The method `example` is no longer a *member-level* covariant use of `A`: it is universally quantified over a fresh type `B` constrained from below by `A`. Inside the trait declaration, `A` itself appears only in the contravariant position of `serialize` and in the lower-bound of `B` (which is also a contravariant position), so the declaration `-A` is preserved.

An implementation, say for `Person`, returns a `Person` value, which is automatically a valid `B` for any `B >: Person`:

```
class PersonSerializer extends Serializer[Person]:
  def serialize(p: Person): String  = ???
  def example[B >: Person]: B       = (new Person(...)).asInstanceOf[B]  // or use the fact that
```

The caller picks `B`; the implementer always has a value typed at `A` (or a subtype) on hand, which fits any `B >: A`. Variance — and the substitution payoff — is restored.